

1. OBJECTIF

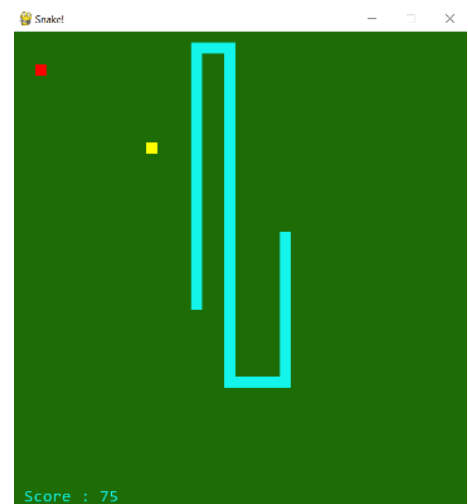
Le but est d'écrire un code python permettant de jouer au snake, jeu décrit ci-dessous :

a) Le cadre fonctionnel

- ✚ Un serpent évolue dans un jardin dont il ne doit jamais sortir (s'il sort, il meurt et le jeu s'arrête). Son corps est une suite contiguë de carrés inscrits dans le jardin.
- ✚ Il peut grandir et accroître son score en mangeant des pommes qui apparaissent aléatoirement dans la fenêtre.
- ✚ Lorsque le serpent avance, le corps suit sa tête. Ainsi le mouvement est entièrement défini par le cheminement de la tête.
- ✚ Il ne doit pas se mordre (sinon il meurt et le jeu s'arrête).
- ✚ Il doit manger de temps en temps sinon il perd progressivement des forces, finit par mourir affamé et le jeu s'arrête.
- ✚ À chaque instant, une seule pomme jaune est à sa disposition dans le jardin, occupant une case aléatoirement située dans le jardin. Lorsque la case occupée par la tête du serpent coïncide avec la pomme, le serpent grandit d'une case et le score est incrémenté.
- ✚ Les pommes se décomposent et elles ont une durée de vie limitée et aléatoire. Elles disparaissent du jardin à l'expiration de cette durée de vie.
- ✚ Les extra-pommes (qui sont rouges) font aussi grandir le serpent, mais ajoutent également un bonus de points à son score. Comme les pommes jaunes, une seule ne peut être disponible à un instant donné. Cependant, elles sont plus rares que les jaunes (il n'y a pas toujours d'extra-pommes dans le jardin) et plus fragiles (elles se décomposent plus vite).
- ✚ Les touches directionnelles serviront pour diriger le (premier) serpent.
si le joueur relâche les flèches de direction, le serpent continue sur son élan.

b) Le cadre graphique

- ✚ la fenêtre graphique est constituée de cases.
- ✚ Le serpent a une couleur uniforme.
- ✚ Les pommes occupent une case.
- ✚ Le score du serpent est affiché dans un bandeau inférieur.
- ✚ Le rendu est analogue à celui-ci-contre :



c) Premiers choix (imposés) de conception

Le jardin est modélisé par une grille. Chaque case ou point de cette grille constitue une unité graphique. (L'implémentation les désignera comme des points)

La progression et le rafraichissement de la fenêtre graphique sont périodiques (la période est ajustable en fonction du niveau du joueur, de l'ordre de 100 ms, dans un fonctionnement « joueur », mais plutôt de l'ordre de 1000 ms dans un fonctionnement « développeur débogueur »).

La vitesse du serpent en déplacement est **d'une case par période rafraichissement**.

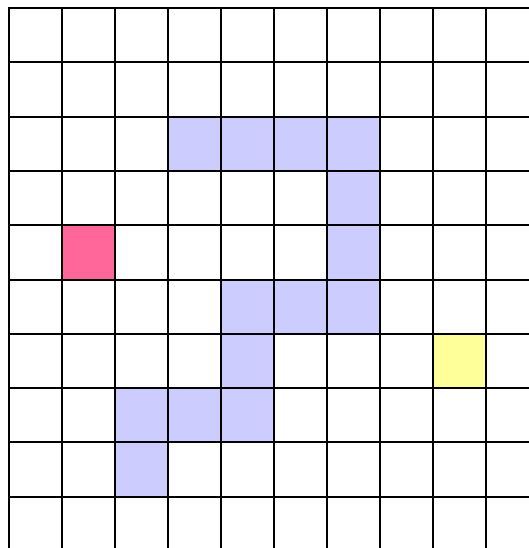
Le déplacement ne peut être qu'horizontal ou vertical (diagonale interdite)

👉 La vitesse est donc définie par une vitesse horizontale de +1, 0 ou -1, et par une vitesse verticale de +1, 0 ou -1, soit un couple prenant les seules valeurs possibles suivantes :

Lorsque la tête avance, chaque partie du corps prend la position de la partie la précédant.

Ainsi les seuls décalages d'une case dans la grille de la tête et de la queue du serpent donnent l'illusion d'un serpent qui avance.

Il suffit donc de définir le corps du serpent par **une liste** de cases dont la tête avance à chaque période selon sa vitesse horizontale ou verticale tandis que la queue est supprimée.



2. QUELQUES OUTILS PYTHON UTILES

👉 Les fichiers suivants sont disponibles sur Moodle :

- 🔗 Le fichier `snake_pygame.py` contient les fonctions utiles permettant la gestion graphique et la gestion du temps. Il n'est a priori pas nécessaire de modifier ces fonctions. Elles permettent la création de la fenêtre graphique et les dessins dans un repère (origine en haut à gauche de la fenêtre, axe des x horizontal orienté vers la droite, et axe des y vertical orienté vers le bas).
- 🔗 Le fichier `snake_classes.py` à compléter, contiendra les fonctions essentielles des classes `Point`, `Pomme` et `Snake`.
- 🔗 Le fichier `snake_jeu.py` à compléter, définit un squelette de la fonction `jouer()`.

3. LES ETAPES DU CODAGE

1. Renommer les fichiers `snake_classes.py` au format `nom1_nom2_snake_classes.py` et `snake_jeu.py` au format `nom1_nom2_snake_jeu.py`.
Importer le fichier `snake_pygame.py` (et le module `random`).
2. Le point est l'unité graphique : une case ou un carré de couleur uniforme et dont la taille est unique, inscrit dans la fenêtre graphique.
Coder les méthodes de la classe `Point` (respecter les *docstrings* de chaque méthode figurant dans le fichier `snake_classes.py`).



Tester toutes les méthodes de la classe à l'aide des tests figurant dans la partie inférieure du fichier.

3. Coder les méthodes de la classe `Pomme` dont on fournit les *docstrings* dans le fichier `snake_classes.py`.



On peut implémenter une pomme en associant un attribut `point`, instance de la classe `Point`, et un attribut `duree` (entier désignant la durée de vie de la pomme avant expiration et disparition du jardin).
D'autres attributs seront peut-être nécessaires.
D'autres méthodes en plus de celles déjà prototypées dans le fichier disponible sur Moodle seront peut-être nécessaires aussi.



Tester toutes les méthodes de la classe (intégrer ces tests au fichier).

4. Coder les méthodes de la classe `Snake` dont on fournit les *docstrings* dans le fichier `snake_classes.py`.



On peut implémenter un serpent en lui associant entre autres un attribut `corps`, liste de `points`, eux-mêmes instances de la classe `Point`.
D'autres attributs seront peut-être nécessaires.
D'autres méthodes seront peut-être nécessaires aussi.



Valider (et éventuellement compléter) les tests des méthodes de la classe (intégrer ces tests au fichier).

5. Dans le fichier `snake_jeu.py`,
 - a. Importer les deux fichiers `snake_pygame.py` et `snake_classes.py` (et le module `random`)
 - b. Coder la fonction `nouvelle_pomme(x_m, y_m, snk, coul)` dont on fournit la *docstring*.
 - c. Compléter la fonction `jouer()` afin qu'elle respecte les spécifications fonctionnelles et graphiques décrites en page précédente.
La probabilité de création d'une pomme rouge à chaque période est ajustable (de l'ordre de 5% dans un fonctionnement « joueur »).
 - d. Jouer et tester.
6. **Pour serpenter plus loin :**
 - a. Implémenter un menu qui demande si l'utilisateur veut des extra-pommes et à quel niveau il veut jouer (entre 1 et 9).
 - b. Modifier le code du module `snake_pygame.py` ainsi que votre code afin de permettre à deux joueurs de jouer simultanément :
 - 🚩 le 1^{er} serpent partira du quart haut et gauche de la fenêtre tandis que le 2^e serpent partira du quart bas et droit.
 - 🚩 Le deuxième serpent est piloté par les clés Q, S, D et Z.
 - 🚩 Les deux scores sont affichés dans le bandeau des scores.
 - 🚩 la collision entre les serpents fait gagner le serpent mordu, la sortie du jardin fait perdre le serpent fautif, et un serpent affamé est perdant.

4. POUR FIXER LES IDEES :

👉 On donne quelques constantes possibles (à intégrer dans le fichier `snake_jeu.py`) pour rendre le jeu raisonnablement « jouable » :

⚠ En phase de debug, ces constantes ont tout intérêt à être modifiées.

```
PERIODE_RAFF_MS=100          #periode de rafraichissement en ms
LEVEL=1                      # niveau 1 : tranquille - niveau 9 : complètement speed

# dimensions de la fenêtre DIM_X (en pxl)* (DIM_Y(en pxl) + taille score (en pxl))
DIM_X, DIM_Y = 40, 40       # nombre de cases (point) en x et en y dans la fenetre
H_SCORE_EN_PXL = 40         # hauteur du bandeau de score en pixels
CASE_EN_PIXEL = Point.taille_pxl # taille d'une case (unite de longueur du snake)

COLOR_SNAKE_0= (19, 244, 234) # joli bleu lagon
PROBA_POMME_ROUGE_POUR100=5   # probabilité en % d'apparition d'une pomme rouge à chaque période
DUREE_POMME_JAUNE_MAX=100     # duree de vie maximale d'une pomme jaune
DUREE_POMME_JAUNE_MIN=50      # duree de vie minimale d'une pomme jaune
REDUCTION_POMME_ROUGE=30      # les pommes rouges disparaissent plus vite que les jaunes
```